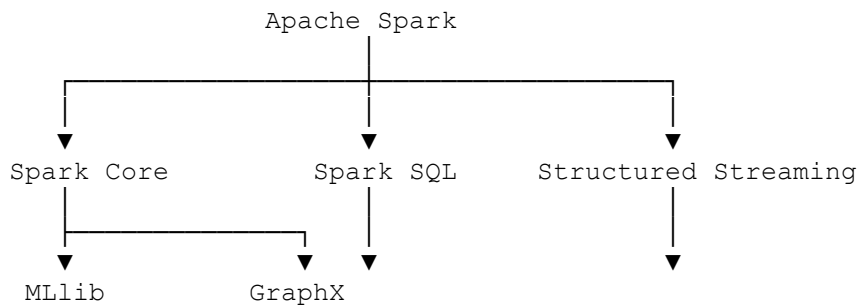


Apache Spark Components

Apache Spark is a **unified analytics engine** designed for large-scale data processing. It consists of several built-in components (libraries), each specialized for a different type of data processing.

Apache Spark Architecture



All these components run on top of **Spark Core**.

1. Spark Core

What is Spark Core?

Spark Core is the **foundation** of Apache Spark. Every Spark application uses Spark Core because it provides the basic functionality required for distributed data processing.

It is responsible for:

- Distributed task execution
- Memory management
- Fault tolerance
- Scheduling
- Job execution
- Resource management
- Input/Output operations
- RDD API

Without Spark Core, none of the other Spark components can work.

Main Features

- RDD (Resilient Distributed Dataset)
- DAG Scheduler
- Task Scheduler
- Fault Recovery using Lineage
- In-memory computation
- Distributed execution

Example

```
rdd = spark.sparkContext.parallelize([1,2,3,4])
```

This operation is handled by Spark Core.

2. Spark SQL

What is Spark SQL?

Spark SQL is the component used for **structured and semi-structured data**.

It allows you to work with:

- DataFrames
- SQL queries
- Tables
- JSON
- CSV
- Parquet
- ORC
- Avro (with package support)

Spark SQL internally uses the **Catalyst Optimizer** and **Tungsten Execution Engine** to optimize query execution.

Main Features

- DataFrame API
- SQL API
- Catalyst Optimizer
- Cost-based optimization
- Schema support
- Column pruning

- Predicate pushdown

Example

```
df = spark.read.csv("employees.csv", header=True)

df.filter(df.salary > 5000).show()
```

or

```
df.createOrReplaceTempView("employees")

spark.sql("""
SELECT *
FROM employees
WHERE salary > 5000
""").show()
```

3. Structured Streaming

What is Structured Streaming?

Structured Streaming is Spark's **stream processing engine**.

It processes **continuous streams of data** while using the same DataFrame and SQL APIs as batch processing.

Examples of streaming sources include:

- Kafka
- Socket streams
- File streams
- Cloud storage events

Features

- Near real-time processing
- Fault tolerance
- Exactly-once processing (with supported sinks)
- Incremental execution
- Scalable distributed processing

Example

```
stream = spark.readStream.format("kafka").load()
```

4. MLlib (Machine Learning Library)

What is MLlib?

MLlib is Spark's distributed machine learning library.

It provides scalable implementations of common machine learning algorithms that run across a Spark cluster.

Algorithms

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Gradient Boosted Trees
- K-Means Clustering
- ALS Recommendation
- PCA
- Feature Engineering

Example

```
from pyspark.ml.classification import LogisticRegression
```

5. GraphX

What is GraphX?

GraphX is Spark's graph processing library.

It is used to analyze data represented as graphs, where:

- Vertices (Nodes)
- Edges (Relationships)

Example use cases:

- Social networks
- Fraud detection

- Recommendation systems
- Network analysis
- Route optimization

Example

Graph:

```
Alice ---- Bob
|
Charlie
```

GraphX can answer questions such as:

- Who has the most connections?
- What is the shortest path?
- Which users belong to the same community?

Note: GraphX is available in **Scala and Java APIs**. It is **not supported in PySpark**. For Python users, graph processing is often done with libraries such as GraphFrames.

How the Components Work Together

Example workflow:

1. Read data using **Spark SQL**.
2. Clean and transform it with **DataFrames**.
3. Train a model using **MLlib**.
4. Process live events using **Structured Streaming**.
5. Analyze relationships using **GraphX**.
6. **Spark Core** manages execution, scheduling, memory, and fault tolerance throughout the workflow.

Summary Table

Component	Purpose	Main API	Common Use Cases
Spark Core	Core distributed processing engine	RDD	Task scheduling, memory management, fault tolerance

Component	Purpose	Main API	Common Use Cases
Spark SQL	Structured data processing	DataFrame, SQL	ETL, analytics, SQL queries
Structured Streaming	Real-time stream processing	DataFrame, SQL	Kafka, event processing, live dashboards
MLlib	Distributed machine learning	ML Pipelines	Classification, regression, clustering
GraphX	Graph analytics	Graph API (Scala/Java)	Social networks, fraud detection, recommendation systems

Interview Questions

Q1. What are the main components of Apache Spark?

The five main components are:

- Spark Core
 - Spark SQL
 - Structured Streaming
 - MLlib
 - GraphX
-

Q2. Which component is the foundation of Spark?

Spark Core is the foundation of Apache Spark. All other Spark components rely on it for distributed execution, scheduling, memory management, and fault tolerance.

Q3. Which Spark component is used most in Data Engineering?

Spark SQL is the most commonly used component because most data engineering workloads involve reading, transforming, joining, aggregating, and writing structured data using DataFrames and SQL.

Q4. Which component processes real-time data?

Structured Streaming is used for processing real-time data streams from sources such as Kafka, sockets, and file streams.

Q5. Which component provides machine learning capabilities?

MLlib provides distributed machine learning algorithms and tools for feature engineering, model training, and prediction.

Interview Summary

- **Spark Core** is the execution engine that provides distributed processing, memory management, scheduling, and fault tolerance.
- **Spark SQL** is used for processing structured data with DataFrames and SQL.
- **Structured Streaming** enables scalable, fault-tolerant processing of real-time data streams.
- **MLlib** provides distributed machine learning algorithms and pipelines.
- **GraphX** is Spark's graph processing library for analyzing relationships between entities. It is available in Scala and Java, while Python users typically use GraphFrames for graph analytics.